Regular Expressions

Why is it called Regex ?
What is Regex ?
Why do we use Regex ?
Where can we use Regex ?
History
    1. Literals
    2. Meta-Characters
        How can we search for one of these characters themselves ?
Standards
    Why do I really have to care about standards ?
Practice [https://regex101.com/]
BRE
    . [Period character / 'ANY'] Or The Dot Character
    Anchors (^) and ($)
        Bracket Expressions ([ ) and ( ])
    Character Classes
    Extended Regex [ERE]
        Alteration (|) OR
        Quantifiers [by7ddo l Quantity]
        Matching count ({) and (})
    Other Commands
    sed & awk TOOLS
    sed
    Syntax
        Substituting Text (The 's' command)
        Selecting the delimiter character
    Using the Global Option
    Using the Matched String (&)
    awk
        Syntax
Lookahead? Pos/Neg
Capturing Group
    More Advanced AWK scripts

More on AWK
Python
grep [Global Regular Expression print]
grep options
Regular Expressions
What is Regular Expressions ??
Regex Are Complicated
grep Command
Literal Vs. Meta-Characters

Why is it called Regex ?

Because **REGULAR EXPRESSIONS** is a *mouthful* EXPRESSION 😀

What is Regex ?

Simply put, regex are symbolic notations used to identify patterns in text .

Why do we use Regex ?

It's **useful** when it comes to the case when you **don't actually remember** a **spelling** of a word or even the **exact expression** you wrote in a **huge file** so , you will need to search for it using some characters that you can remember .

For Example : **Reg[o/u]lar**

Also some words like : favo[**u**]rite , colo[**u**]r , .. etc can be written with / without U.

Where can we use Regex ?

- **Notepad++**
- **IDEs** [Eclipse]
- **Google chrome** [extension]
- **Linux grep command**

    - [file change all characters from lowercase to uppercase/Remove a word from all files in certain directory]
- **Programming languages** : Java , C++ , Python , ...
- **Operating Systems** : **Unix**
- **Built-in** in some languages
    - Perl, Javascript, Ruby, AWK,Tcl
- Can be **imported** in some others
- .NET languages , Java, Python , POSIX C & C++11
- Different **flavors** among them .

History

- It originated in **1956** by **Stephen Cole** .
- **Searching** , **Text editors** and **compilers** .
- It became popular from **1968** in two domains :
    - **Pattern matching** in **text editor** .
    - `Lexical analysis` in a **compiler** .
- After it became popular in editors and compilers
    - [**Ken Thompson**] added this **feature** to the `ed` editor [**first editor**] in Unix .
- Regex are divided into two sets :

1. Literals

- Literal characters are those characters that **represent themselves** in the **search pattern** .
- $grep “error” *.log
    - The letters in “error” are all literal characters

```
a b c d e f g  ...
```

2. Meta-Characters

- Meta characters are those characters that have **special meaning** in the **search pattern** .

```
.
^
$
[
]
{
}
-
?
*
+
(
)
|
\
```

💡 **All other characters** are **literal** characters .

- Normally, it is recommended to put any expression with meta-characters inside a quote, to avoid expansion according the shell rules.

How can we search for one of these characters themselves ?

- **Meta-characters** can be **treated** as **literals** if they are `escaped` **. i.e., preceded by a back slash.**
- Just put '\' before them.
  - Examples: \^, \{, \$, \\
- The back slash can also convert some of the literal character into a meta-characters

- Examples: \d \w

> 💡 We can change a **literal** to a **meta-character** as well

- **Literals** can be treated as **meta-characters** if they are **escaped .**

```
\w

\d
```

Regex is a **combination** between **literals** and **meta-characters**.

Standards

- The **IEEE POSIX** standard classified regex into two sets :
- **BRE** : Basic Regular Expressions [dol lly bystkhdmo m3 el grep kman]
  - fe b2a grep -E aw egrep

```
.
^
$
[
]
*
```

- **ERE** : Extended Regular Expressions [richer set]

```
.
^
$
[
]
*
```

```
(
)
{
}
?
+
|
```

- The tool ‘grep’ uses BRE
- To access ERE, use ‘egrep’ or ‘grep -E’

Why do I really have to care about standards ?

Python does not support the **POSIX** , other language can support so we need to know .

Practice [https://regex101.com/]

BRE

. [Period character / ‘ANY’] Or The Dot Character

- It matches **everything** in the **file** `except` the `new line` .
- Placeholder for `any character` [Not including NULL character]
- For example,
  - $grep “.zip” file.log
    - This searches for a 4 letter text pattern that starts with any character followed by the three letters in ‘zip’
    - May result in: gzip, bzip, gnubzip, rezipped
    - But **will not** result in: zip
  - $grep “\.zip”

a

```
abcd
efgh
ijkl
mnop
qrstu
vwxyz
```

.d

```
abcd
efgh
ijkl
mnop
qrstu
vwxyz
```

.d / **space-d**

```
abc d
efgh
ijkl
mnop
qrstu
vwxyz
```

..s

```
abc d
efgh
ijkl
mnop
qrstu
vwxyz
```

Shorthand character classes [Literals converted to **meta characters]**

`\t` tabs (ASCII 0x09)

`\r` Carriage Return (ASCII 0x0D)

`\xnn` Stands for character with ASCII = nn (\xA9 == @)

`\n` stands for Line Feed (0x0A)

`\w` **is equivalent to** `[a-zA-Z0-9_]` (word character)

`\W` **is equivalent to** `[^a-zA-Z0-9_]`

`\d` **is equivalent to** `[0-9]`

`\D` **is equivalent to** `[^0-9]`

`\s` **is equivalent to** `[whit space + \n]`

`\S` **is equivalent to** `[^white space + \n]`

`\b` **is equivalent to** [anything **except word characters**]

- It matches the position between a word character (like letters, digits, or underscore) and a non-word character (like spaces, punctuation, etc.). It does not match any actual character; It just checks for the boundary between words.

`\B` **is equivalent to** [ay 7aga 8er \b] Vs \W

`Carrot` in a square brackets indicates the `negation` of the **set** .

Example text:
cat
catalog
educate
the cat sat

Regex:
grep -E '\bcat\b' file.txt

✅ Matches:

cat
the cat sat

🚫 Does not match:

catalog
educate

💡 Same idea as grep -w — in fact, -w is like wrapping your pattern with \b au tomatically!

Anchors (^) and ($)

1. **^ [Carrot character]**

- The **string** has to be at the `beginning` **of a line .**
- $grep “^zip” file.txt

^a

abcd
efah
ajal

2. $ [Dollar sign character]

- The string has to be at the `end` of the line .
- $grep “zip$” file.txt
- $grep “^zip$” file.txt
- $grep “^$” file.txt

a$

```
abcd
efaha [fe space hna msh hygebha]
ajala
```

We want to match with the first one .

ab\.png [escape the meta-character]

```
ab.png
abxpng
pic.png
photo.png
p2.png2
```

We want the last one

png$

```
ab.pngx
abxpngy
pic.pngoo
ab.png
```

How to get the last one

^ab

```
zab.pngx
yabxpngy
uabypngoo
ab.png
```

what does it means ?

^ab$

it has to be `alone` in the line .

```
ab
```

^$ [Empty line]

^ab [should be at the beginning of the line]

ab$ [end]

^ab$ [alone]

ab.png [meta-character]

ab\.png [literal]

Boundary [Anchors]

`\bis` [lazm is ableha ay 7aga msh \w]

```
is,is-is!is
is is is
is, was.
is_is_
2is_is4
```

`\bis\b`

```
is,is-is!is
is is is
is, was.
is_is_
2is_is4
```

Bracket Expressions ([ ) and ( ])

Anything inside [] is a `literal` except :

1- `'^'` [Negation] and it has to be at the beginning of the []

2- `'-'` if it comes in the middle it means a range [a-z]

[ab].zip [a lw7da aw b lw7dha] w lazm tb2a mwgoeda

- $grep “[bg]zip” dict.txt [yb2a ablha ya2ma b ya2ma g]

    - Results in : bzip, gzip, aabzip
- Negation
    - $grep “[^bg]zip” dict.txt
    - Will catch words with any character before zip except b or g or NULL character
- Ranges
    - $grep “[a-z]2” dict.txt
        - Will catch words starting with any small letter followed by 2
    - $grep “[a-fA-F]4” dict.txt
    - $grep “[0-9a-zA-Z]”
        - To search for hexa

```
ab.zip
ba.zip
asds.png
sad.tar
```

[^ab].zip

```
ab.zip
ba.zip
asds.zip
sad.zip
```

[^ab].zip

```
ab.zip
ba.zip
asds.tar
sad.mp3
```

[a-z].zip

```
ab.zip
ba.zip
asds.zip
sad.zip
```

[a-z]2

```
abs2
```

[^a-z]2

```
abs02
```

[.]2

```
abs.2
absd2
```

[$]2

```
bs$2
```

Character Classes

- The following character classes can be used,

Class	Description
[[:alnum:]]	
[[:word:]]	
[[:alpha:]]	
[[:digit:]]	
[[:blank:]]	
[[:lower:]]	

[[:upper:]]	
[[:space:]]	
[[:xdigit:]]	Hex digit

Extended Regex [ERE]

Alteration (|) OR

- Matches any line containing `pattern1|pattern2`
- $grep -E “AAA|BBB” file.txt
- We separate the alternation from the rest of the regex using ‘()’
  - a(AAA|BBB)
  - $grep -E “^(AAA|BBB|CCC)” file.txt

cat|dog

```
cat
dog
catdog
```

a|b{2,4}

```
a
bb
bbb
bbbb
bbbbb
```

Quantifiers [by7ddo l Quantity]

```
? [Zero or One]
* []
```

```
+ []
```

1. ‘?’ [Question mark] [Existed or not]

colour|color

colou?r [mwgoda aw msh mwgoda]

```
colour
flavour
favourite
```

2. ‘*’ [asterisk] [Zero or more times]

colou*r

```
colouuuuuur
```

```
colouuuuusur
```

3. ‘+’ [Plus] one or more times

colou+r

```
color [will not match]
```

- [abc]* zero or more times 7rf mn el 3 dol
- [^]+ one or more characters ma3da el spaces

Phone number format [if condition]

We are searching any line starting with a phone number … this can be in the format

```
(nnn) nnnn -nnnn
nnn nnn nnnn
```

.n* Vs .n+

Filtering some wrong formats from excel sheets

```
\(*n*\)*
```

grep -E “^\(?\d\d\d\)? \d\d\d ?-\d\d\d\d” file.txt

grep -E “^\(?[0-9][0-9][0-9]\)? \d\d\d ?-\d\d\d\d” file.txt

```
We're checking for the number formatting
asdmk
(nnn) nnnn -nnnn
nnn nnn nnnn
893 122-8412
(1234) 456 -1234
a(123
andk
ndlaksd
```

```
(123) 456 -1234
123 456 1234
a(123)
ndlaksd
```

- Example,
  - want to check which lines constitute a proper statement, start with a capital letter, followed by any character or space, and ending by a period.
  - $grep -E “^[[::upper::]][[::upper::][::lower::]]*\.$” file.txt

Matching count ({) and (})

```
{
}
```

- Used to match an element specific number of times .

- {n} n times
- {n,m} at least n times , maximum m times .
- {n,} at least n times , to infinity
- {,m} no min , max = m
- Back to Phone Example,
  - $egrep “\(?[:digit:]{3}\)?[:digit:{3}-[:digit:]{4}” file.txt
  - $egrep -i ‘Continued\.*’ file.txt
    - continued....
    - Continued..
  - $grep “[0-9a-fxA-FX]abc” file.txt
  - $egrep ‘fish\|chips\|pies’ file.txt
  - $egrep -i ‘\(cream\|fish\|birthday\|\) cakes’ file.txt
  - $grep ‘[Jj]oe [Bb]loggs’ file.txt
  - $grep -E “colou?r” file.txt
- a{3} a has to be repeated 3 times
- a{1,3} a can be ‘a’ ‘aa’ ‘aaa’

a* [any number of a]

a{1}

```
aaaaa
```

```
abbabbabbbabba
```

a{2}

```
abbaabbaabbbaabbaa
```

a{1,3}

```
aabbaabbaaabbbaabbaaaa
```

a{,1}

```
abbbbbaabbaabbabaa
```

a{1,2}

a{4,}

^\(?^\d{3}\)? \d{3} ?-\d{4}$

```
We're checking for the number formatting
asdmk
(123) 456 -1234
123 456-1234
893 122-8412
(1234) 456 -1234
a(123
andk
ndlaksd
```

Exercise : Contest On Virtual Judge

Other Commands

- A lot of other commands also work with regex such as,
  - The commands ‘find’ and ‘locate’ for finding files
  - The ‘vi’ editor
  - The command less can perform text search using regex
  - Tools that use regex extensively

- sed
- awk

sed & awk TOOLS

sed

- sed stands for **S**tream **Ed**itor.
- sed is used to **modify text files** within a program, script, or from the command line.
- sed is one of the very famous programs to use regex extensively.
- sed is a rich program, and has many options, so we are just providing a quick intro for it in this lecture.

Syntax

- All sed commands are formed into a command string that is passed to sed in the following format.
  - $sed ‘command string’
- The input file that sed works on can be passed through
  - Input redirection
    - $sed ‘command string’ < input-file
  - Pipes
    - $cat input-file | sed ‘command string’
  - The output goes to stdout, in case you want the output in a file, you need to redirect it.
    - $sed ‘command string’ < input-file > output-file

Substituting Text (The ‘s’ command)

- Substituting text is one of the most popular commands of sed

- It searches for all the occurrences of a certain text pattern (using REGEX) and substitute it with different string
- Example:
  - $sed ‘s/day/night/’ <old >new
    - slash is a delimiter
    - Search for day and replace it with night

Selecting the delimiter character

- By default, sed uses the slash as a separator (delimiter) between the command, search pattern, and the replacement pattern.
- If we need to use a slash inside the search pattern or the replacement pattern, we must escape it (precede it with a backslash)
  - $sed ‘s/yes\/no/true\/false/’ <old-file > new-file
- Another option is to use another character as a separator,
  - $sed ‘s:yes/no:true/false:’ <old-file > new-file
  - $sed ‘s|yes/no|true/false|’ <old-file > new-file

Using the Global Option

- sed works on the file line by line
- So this command
  - $sed ‘s/day/night/’ <old-file > new-file
  - Changes one occurrence of the search pattern (day) in each line into the replacement pattern (night)
  - If the search pattern exists multiple times inside the same line, only the first occurrence will be substituted.
- To avoid that use the global flag.
  - $sed ‘s/day/night/g’ < old-file > new-file

Using the Matched String (&)

- Suppose that we want to search for all occurrences of a string, and want to put it inside brackets. We would use the command
  - $sed ‘s/abc/(abc)/g’ <old-file > new-file
- What if we want to use a pattern for our search, then we can use the meta-character & to indicated the matched string
  - $sed ‘s/gr[ae]y/(&)/g’ <old-file > new-file
  - $sed -r ‘s/[0-9]+/(&)/g’ <old-file > new-file
- Another example, let us say we want to repeat any number in the file
  - $sed -r ‘s/[0-9]+/& &/g’ < old-file > new-file
- Note, the last examples have been using ‘-r’ to support Extended REGEX.

awk

- awk is an excellent tool for processing files containing tabulated data (in rows and columns)
- awk also has strong string manipulation functions, so it can search for particular strings and modify the output.
- awk also has support for associative arrays, which are very
- There is a GNU version of AWK called “gawk”
- The name “awk” comes from the names of its inventors.

Syntax

- awk reads the input file line by line
- On each line, it performs a test based on some pattern
- If the test succeeds it performs an ection
- Thus an awk statement would look like:
  - pattern { action }
- Some basic pattern,

- No pattern: perform the action on every line.
- BEGIN: perform the action if this is the beginning of the file
- END: perform the action if this is the end of the file

- Example:
  - BEGIN {print “Now we will start the file”}
  - {print}
  - END {print “That was the end of the file”}

Lookahead? Pos/Neg

Capturing Group

More Advanced AWK scripts

BEGIN {print “File\tOwner”}

{print $9, “\t” ,$3}

END {print “ - DONE -”}

- This will do the following,
  - When the first line of the input file is read, the scripts outputs the string “File” and “Owner” separated by a TAB.
  - After the input file is fully read, the scripts prints “ - DONE - ”
  - Now if you apply this awk statements on the output of
    - $ls -l
    - It should make a list of file names and owner name for all files.

More on AWK

- AWK has a very powerful syntax

- It can be written inside a bash script
- It can define variable, perform mathematical operations
- It also have a strong support for REGEX

Python

It does not have built-in regex so ,

```
import re # search / find / findall
# It only gets the first occurrence
re.search()
#method that takes a regex pattern and a string and searches for that pattern within that string .
#It returns → if found it returns a match object else it returns None

str = 'an example word:cat!!'
match = re.search('word:\w\w\w',str)
print(match) # ... word:cat
print(match.group()) #word:cat
print(type(match)) # class re.Match
print(str(match)) # TypeError ; 'str' object is not callable
print(type(match.group())) # class str
if match:
    print('found',match.group()) ## 'found word:cat'
else:
    print('Not found')

match2 = re.search(r'iii','piiig')
match3 = re.search(r'igs','piiig')

if match2:
    print('found',match2.group())
```

```
else:
    print('Not found')

if match3:
    print('found',match3.group())
else:
    print('Not found') ## 'Not found'

## . = any character but \n
match4 = re.search(r'..g','piiig')

if match4:
    print('found',match4.group()) ## found iig
else:
    print('Not found')

## \d ⟶ any digit [0:9]
## \w ⟶ word characters

match5 = re.search(r'\d\d\d','p123g')
if match5:
    print('found',match5.group()) ## found 123
else:
    print('Not found')

match6 = re.search(r'\w\w\w','@@abcd!!')
if match6:
    print('found',match6.group())  ## found abc
else:
    print('Not found')

#Repetition

## i+ = one or more i's as many as possible
match7 = re.search(r'pi+','piiig')
if match7 :
```

```
    print('found',match7 .group())  ## found abc
else:
    print('Not found')

match8 = re.search(r'i+','piigiiii')
if match8:
    print('found',match8.group())  ## found abc
else:
    print('Not found')

## \s* = zero or more whitespace , newlines

match9 = re.search(r'\d\s*\d\s*\d', 'xx1 2 3xx')
if match9 :
    print('found',match9 .group())  ## found 1 2 3
else:
    print('Not found')

match10 = re.search(r'\d\s*\d\s*\d', 'xx12 3xx')
if match10 :
    print('found',match10 .group())  ## found 12 3
else:
    print('Not found')

match11 = re.search(r'\d\s*\d\s*\d', 'xx123xx')
if match11 :
    print('found',match11 .group())  ## found 123
else:
    print('Not found')

match12 = re.search(r'^b\w+','foobar')
if match12:
    print('found',match12.group())
else:
    print('Not found')
```

```
match13 = re.search(r'b\w+', 'foobar')
if match13 :
    print('found',match13.group())
else:
    print('Not found')

str = 'purple alice-b@google.com monkey dishwasher'
match14 = re.search(r'\w+@\w+', str)
if match14 :
    print('found',match14.group()) # b@google
else:
    print('Not found')
```

How to print the email only

```
match15 = re.search(r'\w+-\w@\w+\.com',str)
if match15 :
    print('found',match15.group()) # b@google
else:
    print('Not found')
```

Grouping/ Substitution

```
match15 = re.search(r'(\w+)-(\w)@(\w+)\.com',str)

print(match15.group())
print(match15.group(1)) #
print(match15.group(2)) #
print(match15.group(3)) #
```

findall function

It finds all matches and returns them as a list of strings with each string representing one match .

```
# suppose that we've a text with many email addresses
# target : alice-b@google.com , bob@abc.com
str2 = 'purple alice-b@google.com blah monkey bob@abc.com blah dishwash
er'
## here re.findall() return a list of all the finded emails

emails = re.findall(r'\w+-*\w+@\w+\.com',str2)
# r'[^\s]+@\w+\.com'
for email in emails :
    print(email)

# alice-b@google.com
# bob@abc.com
```

findall with files

```
'''
For files , instead of looping over a file
f = open('test.txt','r')
string = re.findall(r'regex',f.read())

'''
```

findall with groups

The parenthesis () group can be combined with findall()

if the pattern includes 2 or more parenthesis groups , then instead of returning list of strings , findall() returns a list of tuples .

```
str3 = 'purple alice-b@google.com blah monkey bob@abc.com blah dishwash
er'
```

```
tuples = re.findall(r'\w+-*\w+@\w+\.com',s)
print(type(tuples)) # class list
['alice-b@google.com', 'bob@abc.com']
```

```
str3 = 'purple alice-b@google.com blah monkey bob@abc.com blah dishwash
er'
tuples = re.findall(r'(\w+)-*(\w+)@\w+\.com',s)
print(type(tuples))    # class
print(type(tuples[0])) # class
[('alice','b'),('bo','b')]
```

```
print(re.findall('abc(de|ed)fg(123|321)','abcdefg123 and again abcedfg321 abc
defg123'))
```

substitution

```
re.sub(regex,replacement,str)
#it searches for all the instances of pattern in the gicen string and replaces th
em

str4 = 'purple alice-b@google.com blah monkey bob@abc.com blah dishwas
her'
print(re.sub(r'\w+-*\w+@\w+\.com',r'ab@imt.com',str4))
# purple ab@imt.com, blah monkey ab@imt.com blah dishwasher
```

```
str5 = 'purple alice-b@google.com blah monkey bob@abc.com blah dishwash
er'
print(re.sub(r'(\w+-)*\w+@(\w+)\.com',r'\1@iti.com',str5))
#purple alice-@iti.com, blah monkey @iti.com blah dishwasher
print(re.sub(r'(\w+-)*\w+@(\w+)\.com',r'\1@iti.com',str5))
```

Video record for imt embedded linux group

Google slides Omar Soliman

Online youtube videos

Chapter 19 [Regular Expressions]

grep [Global Regular Expression print]

So we can see that grep has something to do with regex . Inn essence , grep searches text files for text matching a specified regex and outputs any line containing a match to standard output .

```
ls /usr/bin | grep zip
```

This will list all the files in the /usr/bin directory whose names contain the substring zip .

The grep program accepts options and arguments this way , where regex is a regular expression.

```
grep [options] regex [file]
```

grep options

Option	Long option	Description
-i	--ignore-case	
-v	--invert-match	Invert match. Normally , grep prints lines that contain a match. This option causes grep to print every line that does not contain a match .
-c	--count	Print the number of matches (or non-matches if the -v

		option is also specified) instead of the lines themselves .
-l	--files-with-matches	Print the name of each file that contains a match instead of the lines themselves .
-L	--files-without-match	
-n	--line-number	
-h	--no-filename	For multifile searches, suppress the output of filenames.

https://www.youtube.com/watch?v=Cec-2KwgdhU&list=PLWXRxAK4bUzc9gq-W2xWDe9zEaDcowLfs&index=13&ab_channel=LinuxForEmbeddedSystems

Regular Expressions

Introduction to the concept 3shan hoa mwdo3 kber s3b ykhls 3la mra w7da. Bs e7na 3ayzen n go through b3d l7agat lly htb2a eye opener.

What is Regular Expressions ??

- We learned how to do simple text operations (like search on single strings...)
- How about if I want to,
  - Search for string-1 or string-2 ..
  - Search for a string only if it occurs at the beginning of the line
  - Search for a pattern (such as phone number, email, URL, ..)
  - Search for a pattern that have some repetition
  - etc..

    - This means we need a more powerful mechanism to deal with text patterns ..
    - That is the role of Regular Expressions (REGEX)
- We learned to use patterns with filenames via wild cards and other expansions but those expansions were limited to file names.
- Regex use similar techniques but used to general text patterns (so don't confuse the two, cause they have different rules)
- Regex are a very powerful tool for creating text patterns for use by several Linux tools (specially for search in text)
- Used a lot in scripting languages and tools such as 'perl' and 'python'
- One of the main tools using it is grep (the 're' in 'grep' stands for regular expressions).

Regex Are Complicated

- A lot of special cases and forms
- Some of the tools using it, have slight changes to the rules
- Some tools have richer set than the one used in bash (such as perl)
- This lecture in only going to introduce the topic, if interested there are a lot of sources on REGEX
- [ss] to some references.

grep Command

$grep [options]<string/pattern><files>

Search for the string or pattern within the set of files

Option	Long option	Description
-i	--ignore-case	
-v	--invert-match	Invert match. Normally , grep prints lines that contain a match. This option causes

		grep to print every line that does not contain a match .
-c	--count	Print the number of matches (or non-matches if the -v option is also specified) instead of the lines themselves .
-l	--files-with-matches	Print the name of each file that contains a match instead of the lines themselves .
-L	--files-without-match	
-n	--line-number	
-h	--no-filename	For multifile searches, suppress the output of filenames.

Literal Vs. Meta-Characters